SUBSCRIPTION MANAGEMENT · SALESFORCE TO AZURE

Architecture best practices

Knowledge base 02 · Practices, patterns and standards

Module	Subscription Management
Owner	Engineering team
Version	1.0, for review
Date	8 October 2026

Ascendion Confidential

Document control

Document	Subscription Management - KB 02 - Architecture
Subject	Knowledge base 02 · Practices, patterns and standards
Version	1.0
Date	2026-10-08
Status	For review
Owner	Engineering team
Generated by	`src/generators/emit-client-pack.js`
Source	`docs/client-pack/08-architecture.md`
Source blueprint	`49f2c22e6c8f`
Prepared by	Ascendion
Classification	Ascendion Confidential

Contents

1. Purpose and scope
2. Well-Architected pillars
3. Migration patterns
4. Reference architectures we build on
5. Landing zone and governance
6. Identity, integration and data standards
7. References

1. Purpose and scope

The architecture practices and standards we follow when we move a Salesforce module to Azure: the Well-Architected pillars, the migration patterns, the reference architectures we build on, and the standards for identity, integration and data. The target architecture of this module is in the HLD.

2. Well-Architected pillars

Pillar	Best practice	How we apply it
Reliability	Design for failure; set recovery targets (RPO, RTO) per workload	Zone-redundant database with automatic failover, geo-redundant backups, at least two app instances
Security	Zero trust: verify explicitly, least privilege, assume breach	Entra ID sign-in, private endpoints, managed identities, Key Vault, access enforced in the database
Cost optimisation	Pay for what you use; measure before you size	Start small, right-size from measured load, reserved capacity for steady workloads, budgets and alerts
Operational excellence	Everything as code; observe everything	Bicep for infrastructure, CI/CD for every change, Application Insights, alerts and runbooks
Performance efficiency	Scale out, not up; keep reads close to the data	Container Apps scale rules per domain, read replicas for reporting, calculations stored in the database

3. Migration patterns

Pattern	What it is	Why we use it
Bounded contexts with a shared kernel	Split the system into business domains; share only the core customer data	Each Salesforce cloud or module becomes a domain that changes on its own
Modular monolith	One deployable and one database, with strict module boundaries	Transactions where Salesforce had them; split a domain out later without redesign
Transactional outbox	Write a change and its event in one transaction, then publish	No lost or duplicate side effects between domains
Strangler fig	Move one business domain at a time while the old system keeps running	No big-bang cutover; each step is small and reversible
Parallel run (shadow)	Run both systems on the same inputs and compare the results	Differences become tests before anyone relies on the new system
Anti-corruption layer	A Salesforce-compatible interface in front of the new system	Integrations keep working while they move one by one
Change data capture	Changes in the source flow to the target until cutover	The final freeze is short
Expand and contract	Add the new structure, migrate, then remove the old	Schema changes never break a running release

Pattern	What it is	Why we use it
Blue-green release	Deploy to a staging slot, check, then swap	Zero-downtime releases with an instant way back

4. Reference architectures we build on

We start from Microsoft's published reference architectures and adapt them, rather than inventing a design per client. The target design for this module, with its diagrams and sizing, is in the HLD, sections 3, 5 and 7.

Reference architecture	What we take from it
Baseline zone-redundant web application (Azure Architecture Center)	Front Door with WAF, zone redundancy, private endpoints, managed identities
Microservices with Azure Container Apps (Azure Architecture Center)	One environment for many domain services, jobs and event handlers
Azure landing zones (Cloud Adoption Framework)	Subscriptions per environment, hub-and-spoke networking, policy and governance
Well-Architected service guides for App Service and PostgreSQL	Service-level settings for reliability, security and cost

5. Landing zone and governance

- Separate subscriptions (or resource groups) for development, test and production
- Azure Policy for allowed regions, SKUs, tags and mandatory diagnostics
- Role-based access with Privileged Identity Management for administrators
- Hub-and-spoke networking; private endpoints for data services; no public database access
- Microsoft Defender for Cloud and the Microsoft cloud security benchmark as the security baseline

6. Identity, integration and data standards

Area	Standard
User sign-in	OpenID Connect with Microsoft Entra ID; MFA and conditional access from the client tenant
Service-to-service	Managed identities; OAuth 2.0 client credentials for external systems
APIs	REST with OpenAPI 3 descriptions, versioned, published through API Management
Events	CloudEvents on Service Bus, written through a transactional outbox; idempotent consumers
Transport	TLS 1.2 or later everywhere; HTTPS only
Data protection	Encryption at rest with platform keys (customer-managed keys if policy requires); data kept in the chosen region

7. References

Reference	Publisher	Link
Azure Well-Architected Framework	Microsoft	https://learn.microsoft.com/azure/well-architected/
Azure Architecture Center	Microsoft	https://learn.microsoft.com/azure/architecture/
Baseline highly available zone-redundant web application	Microsoft	https://learn.microsoft.com/azure/architecture/web-apps/app-service/architectures/baseline-zone-redundant
Cloud Adoption Framework: landing zones	Microsoft	https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/
Strangler Fig pattern	Microsoft	https://learn.microsoft.com/azure/architecture/patterns/strangler-fig
Anti-corruption Layer pattern	Microsoft	https://learn.microsoft.com/azure/architecture/patterns/anti-corruption-layer
Well-Architected service guide: App Service	Microsoft	https://learn.microsoft.com/azure/well-architected/service-guides/app-service-web-apps
High availability in Azure Database for PostgreSQL	Microsoft	https://learn.microsoft.com/azure/postgresql/flexible-server/concepts-high-availability
Azure Key Vault best practices	Microsoft	https://learn.microsoft.com/azure/key-vault/general/best-practices
Microsoft cloud security benchmark	Microsoft	https://learn.microsoft.com/security/benchmark/azure/overview
Bicep documentation	Microsoft	https://learn.microsoft.com/azure/azure-resource-manager/bicep/overview
StranglerFigApplication	Martin Fowler	https://martinfowler.com/bliki/StranglerFigApplication.html
BoundedContext	Martin Fowler	https://martinfowler.com/bliki/BoundedContext.html
Pattern: Transactional outbox	microservices.io	https://microservices.io/patterns/data/transactional-outbox.html
CloudEvents specification	CNCF	https://cloudevents.io/
Azure Container Apps overview	Microsoft	https://learn.microsoft.com/azure/container-apps/overview
Microservices with Azure Container Apps	Microsoft	https://learn.microsoft.com/azure/architecture/example-scenario/serverless/microservices-with-container-apps
Pub/Sub API	Salesforce	https://developer.salesforce.com/docs/platform/pub-sub-api/overview